

Piano di Progetto – Servizio Gestione Licenze

Stack tecnologico: back-end Node.js + Express, database SQLite, front-end Ionic (fase successiva).

1. Obiettivi del progetto

Realizzare un servizio (per ora lato back-end) che gestisca in modo centralizzato, sicuro e automatizzato il ciclo di vita delle licenze software in ambito B2B.

Obiettivi principali:

- Registrazione clienti con verifica identità OTP e validazione P.IVA/VAT tramite VIES
- Gestione dei quattro tipi di licenza: Trial Demo, mensile, annuale e Licenza Provvisoria
- Avvisi di scadenza automatici e disattivazione licenze tramite job schedulati (node-cron)
- Funzionamento offline tramite file crittografato AES-256 con chiave di decrittazione a scadenza
- Autenticazione sicura separata per client e fornitore (JWT jsonwebtoken + refresh token rotation)
- Comunicazione con ERP fornitore tramite GET ALARM (evita polling continuo)
- Messaggi automatici email e in-app in italiano e inglese tramite sistema template con placeholder
- Monitoraggio attività client con notifica inattività al fornitore
- Architettura predisposta per futura estensione multi-fornitore e front-end Ionic

2. Condizioni — cosa non è incluso in questo progetto

- Front-end: rimandato a una fase successiva — sarà sviluppato in Ionic
- Nessuna gestione del pagamento: il fornitore gestisce i pagamenti in autonomia; il servizio riceve solo la notifica dell'esito
- Nessuna gestione multi-fornitore attiva: l'architettura è predisposta ma non implementata in questa versione
- Nessuna implementazione OAuth2: autenticazione fornitore tramite API key + JWT; OAuth2 è un'evoluzione futura
- Nessuna gestione clienti extra-UE in questa fase: VIES copre solo paesi UE
- Nessuna logica di fatturazione: il servizio registra l'evento comunicato dal fornitore, non genera fatture
- Nessuna integrazione con sistemi di posta configurabili dall'esterno: SMTP/provider configurato a livello infrastrutturale
- Lo sviluppo della libreria client è responsabilità del fornitore; questo servizio fornisce solo gli endpoint

3–5. TO DO, Tempistiche e GAP

I TO DO sono in ordine logico di implementazione. Desiree e Andrea lavorano in parallelo. La durata totale è di **1 mese** (18 giorni lavorativi: settimana 1 = 3 giorni partendo da mercoledì, settimane 2–4 = 5 giorni ciascuna).

Distribuzione parallela per settimana:

Settimana	Desiree	Andrea
Sett. 1 (3 gg)	TD-02 Schema DB (2 gg) + TD-04 Prodotti (1 gg)	TD-01 Setup infrastruttura (3 gg)
Sett. 2 (5 gg)	TD-06 Verifica licenza (2 gg) + TD-08 Attivazione (2 gg) + inizio TD-11 (1 gg)	TD-03 Auth fornitore (2 gg) + TD-05 Registrazione+OTP+VIES (3 gg)
Sett. 3 (5 gg)	TD-11 fine (1 gg) + TD-12 Test (3 gg) + supporto TD-09 (1 gg)	TD-05 fine (1 gg) + TD-07 Sincronizzazione (2 gg) + TD-09 Licenza Provvisoria (2 gg)
Sett. 4 (5 gg)	TD-12 fine (2 gg) + buffer (3 gg)	TD-10 Job schedulati (3 gg) + buffer (2 gg)

TD-01 — Setup infrastruttura e progetto

Campo	Valore	Dettaglio
Assegnato	Andrea	
Descrizione		Inizializzazione progetto Node.js con Express, configurazione SQLite tramite better-sqlite3 struttura cartelle, variabili d'ambiente (.env con dotenv).
Stima	3 Settimana gg 1	
Dipendenze	Nessuna	

Output atteso	Server Node.js avviabile in locale, DB SQLite inizializzato, variabili d'ambiente configurate
---------------	---

GAP:

- Scelta ORM/query builder per SQLite (es. Knex.js, Sequelize, Drizzle ORM, o better-sqlite3 nativo)
- Strategia per le migrazioni DB (es. Knex migrations, o script SQL eseguiti all'avvio)

TD-02 — Schema DB e migrazioni

Campo	Valore	Dettaglio
Assegnato	Desiree	

Descrizione		Creazione di tutte le tabelle tramite script di migrazione: products, clients, otp_codes, licenses, modules, license_mod client_tokens vendor_token vendors, client_activity email_templates messages, alarm_logs, rate_limits, otp_attempts idempotency vendor_events vendor_generation client_billing, event_sent_logs api_logs, security_alerts license_history Seed di email_templates con tutti i template in italiano e inglese.
Stima	2 gg 1	Settimana
Dipendenze		TD-01 parallelo con TD-01

Output atteso		DB completo con tutte le tabelle e template messaggi pre-caricati
---------------	--	---

GAP:

- Configurazione sistema di migrazione scelto in TD-01 (es. Knex.js migrations)

TD-03 — Autenticazione fornitore (F1, F2)

Campo	Valore	Dettaglio
Assegnato		Andrea

Descrizione	Endpoint F1 (login con API key → JWT + refresh token) e F2 (rinnovo con rotation). Middleware Express riutilizzabile per tutti gli endpoint fornitore. JWT con jsonwebtoken hashing API key con bcryptjs.
Stima	2 Settimana gg 2
Dipendenze	TD-01, TD-02
Output atteso	Fornitore autenticabile JWT valido, refresh token rotation, middleware riutilizzabile

GAP:

- Configurazione jsonwebtoken per generazione e verifica JWT
- Strategia revoca token (blacklist in SQLite o campo revoked)

TD-04 — Registrazione prodotti (F6)

Campo	Valore	Dettaglio
Assegnato		Desiree
Descrizione		Endpoint F6 per registrazione prodotti. Validazione unicità product_key.
Stima	1	Settimana gg 1
Dipendenze		TD-02
Output atteso		Prodotti registrabili dal fornitore autenticato

GAP:

- Nessun GAP significativo previsto

TD-05 — Registrazione cliente e verifica OTP (C1, C2, C3)

Campo	Valore	Dettaglio
Assegnato		Andrea DO con più GAP

Descrizione

C1
con
validazione
formale
P.IVA
(regex
per
paese)
e
chiamata
VIES
(axios
o
node-fetch
per
REST/SOAP
UE),
restituisce
client_id
per
e
chiamate
successive.
C2
(verifica
OTP
tramite
client_id
→
attivazione
trial
+
license_key
con
crypto.randomE
+
offline_token
AES-256-GCM
tramite
modulo
crypto
nativo
Node.js
+
JWT
client,
lockout
30
min
dopo
3
tentativi
falliti),
C3
(nuovo
OTP
tramite
client_id).
invio
email
tramite
Nodemailer.

Stima	4	Settimana gg 2–3
Dipendenze	TD-03, TD-04	
Output atteso		Cliente registrabile, email OTP inviata, trial attivata, file offline generato, JWT client emesso

GAP:

- Integrazione con API VIES tramite axios (REST) o strong-soap (SOAP) — gestione timeout e fallback
- Generazione file crittografato AES-256-GCM con modulo crypto nativo Node.js (disponibile senza dipendenze esterne)
- Configurazione Nodemailer con provider SMTP scelto
- Gestione del caso VIES temporaneamente non disponibile (vat_verified = false con warning)

TD-06 — Verifica licenza e poll messaggi (C4, C5, C6)

Campo	Valore	Dettaglio
Assegnato		Desiree

Descrizione	C4 (verifica stato licenza: se scaduta lo comunica al client ma non aggiorna il DB — delegato al job LICENSE_EXPIRATION) C5 (polling in-app + aggiornamenti client_activity) C6 (rinnovo JWT client con rotation), C7 e C7b (cambio email con verifica OTP sulla nuova email). Middleware autenticazione client Express.
Stima	2 Settimana gg 2
Dipendenze	TD-03

Output atteso	Libreria client in grado di verificare licenza, ricevere messaggi in-app, rinnovare token e cambiare email
---------------	--

GAP:

- Nessun GAP significativo previsto

TD-07 — Sincronizzazione fornitore (F3, F4, O1)

Campo	Valore	Dettaglio
Assegnato		Andrea

Descrizione	F3 (recupero nuove iscrizioni con paginazione) F4 (conferma), O1 (GET ALARM uscente con axios in Node.js, eseguito esclusivamen dai job schedulati — mai in real-time da C2; retry fino a 3 tentativi con email fallback GET_ALARM al fornitore). Log in alarm_logs.
Stima	2 Settimana gg 3
Dipendenze	TD-05

Output atteso	Sincronizzazione fornitore funzionante, GET ALARM inviato e loggato
---------------	--

GAP:

- Gestione retry GET ALARM con backoff esponenziale (tentativi in caso di ERP non disponibile)

TD-08 — Attivazione licenze a pagamento (F5)

Campo	Valore	Dettaglio
Assegnato	Desiree	
Descrizione		F5: attivazione licenza mensile/annua con header Idempotency obbligatorio (UUID, TTL 24h), supporto campo is_provisiona disattivazione precedente, rigenerazione offline_token AES-256-GCM invio messaggi.
Stima	2 Settimana gg 2	
Dipendenze	TD-06	

Output atteso		Licenza a pagamento attivabile, file offline rigenerato, messaggi inviati
---------------	--	---

GAP:

- Nessun GAP significativo previsto

TD-09 — Licenza Provvisoria, billing e revoca (F7, F8, F9)

Campo	Valore	Dettaglio
Assegnato		Andrea

Descrizione	F7/invoice (emissione fattura → Licenza Provvisoria + sospensione avvisi + rigenerazione offline_token), F8/invoice (conferma pagamento → licenza definitiva + riattivazione avvisi). Gestione scadenza automatica provvisoria senza avvisi. Nuovi endpoint: F7/billing (salvataggio dati fatturazione cliente: PEC, SDI, IBAN), F8/revoke (revoca licenza per mancato pagamento, status → revoked + notifica in-app), F9 (rotazione API key vendor con grace period 1h).
-------------	--

Stima	3	Settimana gg 3
Dipendenze	TD-07	
Output atteso		Flusso Licenza Provvisoria completo, sospensione, avvisi corretta, endpoint billing/revoked funzionanti

GAP:

- Logica di sospensione avvisi legata allo stato provisional — flag da gestire nei job schedulati

TD-10 — Job schedulati (node-cron)

Campo	Valore	Dettaglio
Assegnato	Andrea	

Descrizione	Job node-cron per: avvisi scadenza Trial/mensile disattivazione automatica, rilevamento inattività client 7+ giorni, retry GET ALARM (job ALARM_RETRY max 3 tentativi). Configurazione job tramite tabelle vendor_event e vendor_generazione (abilita/disabilita) per evento, frequenza configurabile Anti-duplicati tramite event_sent_increment GET ALARM per eventi di scadenza. Tutti i job idempotenti.
Stima	3 Settimana gg 4
Dipendenze	TD-09

Output atteso		Tutti i job attivi e idempotenti, avvisi inviati nei tempi corretti
---------------	--	---

GAP:

- Configurazione node-cron all'avvio del server Express
- Gestione timezone nei timestamp SQLite per evitare errori di calcolo scadenze (libreria day.js o date-fns)
- Persistenza dello stato dei job in caso di restart (verifica idempotenza tramite stato nel DB)

TD-11 — Sistema messaggi e template

Campo	Valore	Dettaglio
Assegnato		Desiree

Descrizione	Sistema di generazione messaggi da email_template con sostituzione placeholder tramite Handlebars o template literals. Invio email con Nodemailer e creazione record in-app in messages. Copertura di tutti gli event_code: REGISTRAT WELCOME, TRIAL_EXPI MONTHLY_F ANNUAL_EX LICENSE_E LICENSE_A PROVISION. PROVISION. REGISTRAT CLIENT_INA
Stima	2 Settimana gg 2–3
Dipendenze	TD-06

Output atteso	Tutti i messaggi generati correttamente con placeholder sostituiti in italiano e inglese
---------------	--

GAP:

- Nessun GAP significativo previsto

TD-12 — Test, bugfix e documentazione finale

Campo	Valore	Dettaglio
Assegnato	Desiree	
Descrizione		Esecuzione completa della suite di test (Jest + Supertest), correzione bug, documentazione API tramite Swagger UI (swagger-jsdoc + swagger-ui) e verifica integrazione end-to-end con ERP fornitore.

Stima	3 Settimana gg 3-4 + 2 gg fine sett.4
Dipendenze	TDIntegrazione TDProgressiva durante sett. 3-4
Output atteso	Servizio testato, stabile, documentato (Swagger UI) e pronto per il collaudo

GAP:

- Documentazione Swagger in Node.js richiede configurazione manuale (swagger-jsdoc)
- Disponibilità ERP fornitore per test di integrazione reale

6. Analisi dei rischi

Rischi suddivisi per categoria con livello (Alto/Medio/Basso) e misure di mitigazione.

6.1 Privacy e protezione dei dati (GDPR)

Rischio	Livello	Mitigazione
Dati anagrafici clienti (P.IVA, email, ragione sociale) soggetti al GDPR	Alto	Cifratura dati a riposo nel DB SQLite, HTTPS obbligatorio su tutti gli endpoint, policy di data retention da concordare, valutare nomina DPO
Email usate per comunicazioni non autorizzate	Medio	Usate esclusivamente per le comunicazioni di servizio previste; nessun marketing senza consenso esplicito

Log (alarm_logs, client_activity_logs) contengono dati temporali sensibili	Medio	Definire policy di retention (es. cancellazione automatica dopo 12 mesi)
--	--------------	--

6.2 Sicurezza

Rischio	Livello	Mitigazione
Furto API key fornitore	Alto	API key salvata con bcryptjs nel DB, trasmessa solo via HTTPS, revocabile e rinnovabile immediatamente
Brute-force su endpoint OTP (C1, C3)	Alto	Rate limiting con express-rate-limit: max 5 tentativi per IP in 15 minuti; OTP scade in 15 minuti e invalidato dopo primo utilizzo
JWT intercettato o riutilizzato	Medio	JWT di breve durata (60 sec), refresh token rotation con jsonwebtoken, revoca in caso di anomalie
SQL injection tramite campi liberi	Medio	Knex.js (o ORM scelto) parametrizza automaticamente le query; validazione input con joi o zod
File crittografato offline manomesso	Medio	Il file AES-256-GCM include tag di autenticazione — la libreria client verifica l'integrità prima dell'uso
Attacchi DDoS sugli endpoint pubblici	Medio	Rate limiting per IP con express-rate-limit, validazione formale prima di operazioni costose

6.3 Perdita di dati

Rischio	Livello	Mitigazione
Corruzione o perdita del file SQLite	Alto	Backup automatici giornalieri su storage separato, test periodici di restore
Perdita license_key da parte del cliente	Medio	Recuperabile dal fornitore tramite vat_number + product_key con intervento manuale — processo da documentare

Invio duplicato di messaggi da job sovrapposti	Medio	Job node-cron idempotenti: verificano stato DB prima di inviare; campi come inactivity_notified_at prevengono duplicati
--	--------------	---

6.4 Dipendenze esterne

Rischio	Livello	Mitigazione
VIES temporaneamente non disponibile	Medio	Timeout gestito: cliente registrato con vat_verified = false e nota; verifica ripetibile successivamente
Provider email non consegna email OTP	Alto	Retry automatico con backoff tramite Nodemailer, logging errori, possibilità di re-inviare OTP tramite C3
ERP fornitore non risponde al GET ALARM	Medio	Logga l'esito in alarm_logs senza bloccare il flusso; fornitore può fare polling manuale su F3
Cambio API VIES da parte della Commissione UE	Basso	Chiamata VIES isolata in modulo separato per facilitare aggiornamenti futuri

6.5 Rischi di progetto

Rischio	Livello	Mitigazione
Requisiti che cambiano durante lo sviluppo	Medio	Architettura modulare predisposta all'estensione; modifiche concordate per iscritto prima dell'implementazione
Indisponibilità ERP fornitore per test integrazione	Medio	Creare mock ERP per test autonomi con Jest; test reali pianificati con fornitore prima di TD-12
Sottostima tempi TD-05 (VIES + AES-256)	Alto	TD-05 ha più GAP tecnici; se i GAP non risolti entro 2 giorni scalare parte del lavoro su Desiree
Lavoro in parallelo: dipendenze tra TO DO	Medio	Desiree dipende dai moduli di Andrea (auth, OTP) — coordinamento quotidiano necessario nella sett. 2

7. Piano di test

Test organizzati per TO DO. Ogni blocco viene eseguito al completamento del TO DO corrispondente prima di procedere. Framework: **Jest** per unit e integration test, **Supertest** per i test degli endpoint HTTP.

7.1 Tipologie di test

- Unit test (Jest): singole funzioni (generazione OTP, crittografia AES-256-GCM, sostituzione placeholder Handlebars, generazione license_key)
- Integration test (Jest + Supertest): endpoint → SQLite → risposta
- End-to-end: flusso completo dalla registrazione alla scadenza
- Test di sicurezza: token scaduto rifiutato, rate limiting express-rate-limit attivo, brute-force bloccato
- Test di regressione: suite completa pre-SAL in TD-12

7.2 Test per TO DO

Test post TD-01 (Andrea — Sett. 1)

- Server Node.js avvia senza errori in locale
- Variabili d'ambiente caricate correttamente tramite dotenv

Test post TD-02 (Desiree — Sett. 1)

- Tutte le tabelle esistono con campi corretti (query PRAGMA table_info in SQLite)
- Vincoli UNIQUE funzionano (inserimento duplicato product_key rifiutato)
- email_templates contiene tutti gli event_code in italiano e inglese
- Unit test: funzione sostituzione placeholder Handlebars funziona con tutti i valori

Test post TD-03 (Andrea — Sett. 2)

- F1 con API key valida restituisce JWT + refresh token
- F1 con API key non valida restituisce 401
- F2 con refresh token valido restituisce nuovo JWT e nuovo refresh token
- F2 con refresh token scaduto restituisce 401
- Il vecchio refresh token non è più valido dopo la rotation
- Endpoint protetto (es. F3) restituisce 401 senza JWT
- Unit test (Jest): funzione generazione JWT produce claims corretti con jsonwebtoken

Test post TD-04 (Desiree — Sett. 1)

- F6 registra prodotto e restituisce 201
- F6 con product_key duplicata restituisce 409
- F6 senza JWT restituisce 401

Test post TD-05 (Andrea — Sett. 2-3)

- C1 con dati validi crea cliente pending e restituisce 201 con otp_sent, client_id e vat_verified
- C1 con product_key inesistente restituisce 404
- C1 con P.IVA già associata a stessa product_key restituisce 409
- C1 con P.IVA non valida restituisce 422 con invalid_vat
- Email OTP ricevuta all'indirizzo fornito (test con Nodemailer + casella reale)
- C2 con OTP corretto attiva il cliente, crea licenza trial, restituisce license_key + JWT + offline_token
- C2 con OTP errato restituisce 400
- C2 con OTP scaduto restituisce 410
- C3 invalida OTP precedente e ne genera uno nuovo
- offline_token AES-256-GCM decrittabile con la chiave restituita (test con modulo crypto Node.js)
- Unit test: crypto.randomBytes produce valori sempre diversi (unicità license_key)
- Unit test: funzione AES-256-GCM: file decrittabile con chiave corretta, non decrittabile con chiave errata
- Test sicurezza: 6 tentativi C1 dallo stesso IP in 1 ora vengono bloccati da express-rate-limit

Test post TD-06 (Desiree — Sett. 2)

- C4 con JWT e license_key validi restituisce stato active con moduli e scadenza
- C4 con JWT scaduto restituisce 401
- C4 restituisce status expired se la licenza è scaduta, senza aggiornare il DB (delegato al job)
- C5 restituisce messaggi in coda e li marca come consegnati
- C5 non restituisce messaggi già consegnati alla chiamata successiva
- C5 aggiorna last_seen_at in client_activity_logs
- C6 rinnova JWT e ruota refresh token
- C7 invia OTP alla nuova email e restituisce 200 con otp_sent
- C7 con email uguale a quella corrente restituisce 400 EMAIL_SAME_AS_CURRENT
- C7b con OTP corretto aggiorna contact_email nel DB e restituisce 200 con new_email

Test post TD-07 (Andrea — Sett. 3)

- F3 restituisce nuove iscrizioni con vendor_synced = false
- F4 marca le iscrizioni come sincronizzate
- F3 dopo F4 non restituisce le iscrizioni confermate
- O1 viene inviato dal job NEW_REGISTRATION (non in real-time da C2) con alarm_code = NEW_REGISTRATION
- L'esito del GET ALARM viene salvato in alarm_logs

Test post TD-08 (Desiree — Sett. 2)

- F5 attiva la nuova licenza e disattiva la precedente (deactivated_at valorizzato)
- F5 restituisce nuovo offline_token e scadenza
- F5 con Idempotency-Key già elaborata restituisce risposta dalla cache con _cached: true

- Cliente riceve email e messaggio in-app di conferma
- F5 con date incoerenti restituisce 422

Test post TD-09 (Andrea — Sett. 3)

- F7 attiva Licenza Provvisoria se licenza in scadenza o scaduta
- F7 restituisce nuovo offline_token
- Cliente riceve email e in-app di attivazione provvisoria
- Avvisi di scadenza non inviati durante Licenza Provvisoria
- F8 disattiva provvisoria e attiva definitiva con nuovo offline_token
- Cliente riceve email e in-app di conferma pagamento
- Scadenza automatica Licenza Provvisoria genera solo messaggio in-app PROVISIONAL_EXPIRED
- F7/billing salva dati fatturazione cliente (PEC, SDI, IBAN) e restituisce 200
- F7/billing con billing_country = IT e senza pec_email né sdi_code restituisce 422
- F8/revoke imposta status = revoked, invalida offline_token e genera notifica in-app al cliente
- F8/revoke su licenza già revocata è no-op e restituisce 200 (idempotenza)
- F9 genera nuova API key, la vecchia rimane valida per 1h (grace period), poi viene revocata

Test post TD-10 (Andrea — Sett. 4)

- Job avviso Trial invia email + in-app a 7, 3 e 1 giorno prima
- Job non invia avvisi se licenza è in stato provisional
- Job disattivazione automatica aggiorna status a expired il giorno della scadenza
- Job inattività rileva client senza C5 da 7+ giorni e invia email al fornitore
- inactivity_notified_at previene invii duplicati
- Job eseguiti due volte di fila non producono effetti doppi (idempotenza)
- event_sent_log previene invii duplicati per lo stesso evento nella stessa giornata
- Disabilitando un job in vendor_event_config (enabled = false) il job non viene eseguito
- Job ALARM_RETRY riprova O1 fino a 3 volte; al 3° fallimento invia email GET_ALARM_FALLBACK al fornitore

Test post TD-11 (Desiree — Sett. 2-3)

- Tutti gli event_code producono il messaggio corretto nella lingua giusta
- Tutti i placeholder Handlebars vengono sostituiti correttamente
- Messaggi in-app non consegnati restano in coda e vengono restituiti da C5
- Email effettivamente ricevute (test con casella reale)
- Unit test: sostituzione placeholder gestisce correttamente valori mancanti o undefined

Test pre-SAL finale (TD-12 — Desiree — Sett. 3-4)

- Esecuzione intera suite Jest dalla TD-01 alla TD-11

- End-to-end: registrazione → OTP → trial → avviso scadenza → Licenza Provvisoria → pagamento → licenza definitiva → scadenza
- End-to-end offline: attivazione → decrittazione offline_token AES-256-GCM → verifica validità senza connessione
- Test carico leggero: 10 registrazioni simultanee senza conflitti nel DB SQLite
- Verifica che tutti gli endpoint restituiscano i codici HTTP corretti in tutti gli scenari di errore
- Swagger UI (/api-docs) mostra correttamente tutti gli endpoint con schema
- Verifica corretta gestione timezone nei TIMESTAMP SQLite (libreria day.js)

8. Riepilogo stima tempi

Durata totale: **1 mese** — 18 giorni lavorativi (settimana 1 = 3 giorni partendo da mercoledì, settimane 2–4 = 5 giorni). Desiree e Andrea lavorano in parallelo.

#	TO DO	Stima (gg)	Se	Asignato
01	Setup infrastruttura Node.js + Express + SQLite	3	Se	Andrea 1
02	Schema DB + migrazioni	2	Se	Desiree 1 parallelo con TD-01
03	Auth fornitore JWT jsonwebtoken (F1, F2)	2	Se	Andrea 2
04	Registrazione prodotti (F6)	1	Se	Desiree 1
05	Registrazione cliente + OTP + VIES + AES	4	Se	Andrea 2–3 GAP
06	Verifica licenza + poll messaggi (C4, C5, C6)	2	Se	Desiree 2
07	Sincronizzazione fornitore (F3, F4, O1)	2	Se	Andrea 3
08	Attivazione licenze a pagamento (F5)	2	Se	Desiree 2
09	Licenza Provvisoria, billing e revoca (F7, F8, F9)	3	Se	Andrea 3
10	Tutti schedulati node-cron	3	Se	Andrea 4
11	Sistema messaggi + template Handlebars	2	Se	Desiree 2–3

TD-01	Test Jest, bugfix, Swagger docs	3+2	Set	Desiree	
			3-4		
	Effort Desiree	10+2 gg			TD-02,04,06,
	Effort Andrea	15 gg			TD-01,03,05,
	Durata totale	18 gg	1	Lavoro	
			me	in	
				parallelo	

Nota: i giorni di effort totale (Desiree ~12 gg + Andrea 15 gg) sono superiori ai 18 giorni di calendario grazie al lavoro in parallelo. Il buffer è distribuito negli ultimi giorni della settimana 4.